Analysis Study of a Sentiment Classification Model

Tingyu Shi
t5shi@ucsd.edu

1 Introduction

The NLP task for this analysis study is sentiment classification. Sentiment classification can be useful in many ways. Here, I present 2 applications of sentiment analysis. First, companies can use sentiment analysis to monitor customer opinions about their products, services, etc., to know what to improve. Also, sentiment analysis can be used in recommendation systems to understand user preferences better in order to provide more personalized and relevant suggestions.

The model used in this analysis study is a RoBERTa-base model trained on about 124M tweets from January 2018 to December 2021, and fine-tuned for sentiment analysis with the TweetEval benchmark. (Barbieri et al., 2020) and (Loureiro et al., 2022) are two relevant papers. The two related source codebases are timelms and tweeteval.

The approaches of analyzing this sentiment classification model include observation of failed data samples, benchmark dataset perturbation to test model robustness and designing adversarial data samples for model stress test.

The results indicate that this model does not perform well when there are (1)negation words, (2) typographical error(typos) and (3)irony in the sentences. Also, I found that this model presents certain (4)biases towards underrepresented people, countries, etc. Therefore, in total 4 limitations of this model were identified.

2 Dataset

The main dataset I used is TweetEval. This dataset has 45.6k training data samples, 2k validation data samples, and 12.2k testing data samples. Each data sample contains a sentence and the corresponding sentiment label. There are only 3 possible values for the sentiment label, which are positive, neutral, or negative. I will use $\{\texttt{POS}, \texttt{NEU}, \texttt{NEG}\}$ for the rest of the report. Since the model is fine-tuned using the training and validation sets, I only used the testing set for the analysis study. The overall testing set accuracy is 72.18%. Table 1 shows the per-class precision and recall of the testing set.

	NEG	NEU	POS
Precision	68.95%	75.66%	71.01%
Recall	80.66%	65.75%	74.06%

Table 1: Per-Class Precision and Recall on Testing Set

I also used Equity Evaluation Corpus(EEC), which consists of 8,640 English sentences carefully chosen to tease out biases towards certain races and genders. Besides, I also came up with some sentences and showed that the model has biases towards ages, people with disabilities and people with different socioeconomic status. In addition, GPT-4 was used to create 100 ironic sentences with negative sentiments to see if the model can perform well with irony.

The model's input is a sentence, and its output is one of the sentiments mentioned above. The following is an example input/output pair:

- Input: *So disappointed in wwe summerslam! I want to see john cena wins his 16th title*
- Output: NEG

3 Analysis Approach

The simplest analysis approach is observing the sentences the model didn't classify correctly. Through the observation, I found that the model has difficulty when handling negation words. Also, based on my personal experience, it's common to make typos when writing reviews, comments, or tweets. Therefore, I intentionally added

some typos to the testing set to see if the model can perform well with one or two typos in the sentences. In addition, according to some literature about sentiment analysis (Wankhade et al., 2022), models may not perform well with ironic text and may have potential biases. Therefore, with the help of GPT-4, I came up with some adversarial data samples for the stress test.

4 Errors and their Categorization

4.1 Negation Words

Common negation words include *not*, *don't*, *didn't*, etc. Also, some prefixes and suffixes express negation meanings such as *un-*, *dis-*, *-less*, etc. However, just because a sentence contains negation, we cannot infer the overall sentiment is negative. This analysis study considers two negation words, *don't* and *not*, to show the limitation of this model. Among 12.2k data samples in the testing set, 236 data samples contain word *don't*, and 581 data samples contain word *not*.

	NEG	NEU	POS
Precision	67.07%	68.00%	54.55%
Recall	85.27%	39.08%	60.00%

Table 2: Per-Class Precision and Recall of sentences that contain *don't*

	NEG	NEU	POS
Precision	69.36%	71.42%	55.00%
Recall	85.27%	51.18%	62.85%

Table 3: Per-Class Precision and Recall of sentences that contain *not*

If we run models on sentences that contain *don't* and *not*, the accuracies are 66.10% and 69.01%, which are not too low compared with the overall accuracy. Table 2 and table 3 showed the per-class precision and recall of sentences that contain *don't* and *not*. We can find out the precision and recall of sentences with positive and neutral sentiments dropped a lot compared with overall per-class precision and recall showed in table 1.

More specifically, 107 sentences contain *don't* and have positive or neutral sentiments; the model classified 54(50.4%) out of them as negative. 289 sentences contain *not* and have positive or neutral sentiments; the model classified 110(38.06%) out of them as negative. This can indicate that the model easily thinks negation words represent negative sentiments. Here, I show 3 wrong predictions when the text has *don't* and 3 wrong predictions when the text has *not*.

3 Wrong Predictions when text contains *don't*:

1. Input text: *@user I don't have time to prepare vegan foods for myself everyday. After my liquid diet, I am returning to vegetarianism.*
   True Label: `NEU` – Model Prediction: `NEG`

2. Input text: *I really don't care. Final is on Tuesday and I'm watching westworld oops*
   True Label: `POS` – Model Prediction: `NEG`

3. Input text: *@user @user I have a great life & I'll be fine. Many of friends dont feel that security. People like Steve Bannon don't help.*
   True Label: `POS` – Model Prediction: `NEG`

3 Wrong Predictions when text contains *not*:

1. Input text: *"Conservatives" not being very conservative if they think 'written agreements' overrides the Constitution.#AusPol*
   True Label: `NEU` – Model Prediction: `NEG`

2. Input text: *@user Even though i have my personal beliefs about abortion, i also know that i am not in charge of other women's rights.*
   True Label: `NEU` – Model Prediction: `NEG`

3. Input text: *@user @user @user there will not be a wall, Obamacare is staying, Trump is pro choice, he will not drain the swamp.*
   True Label: `NEU` – Model Prediction: `NEG`

4.2 Typos

Typos are common occurrences in written text, especially in informal settings such as social media posts, user reviews, and online comments. These errors include misspellings, the omission or addition of characters, etc. To test the model's robustness when there are typos, I intentionally put 1 or 2 typos in 12.2k sentences from the testing set.

I used a 2-step procedure to inject typos into sentences. The first step is randomly selecting 1 or 2 characters from the sentence. The second step is replacing selected characters with characters nearby on the keyboard. For example, if "a" is selected in the first step, then the program will replace "a" with one of the characters from {"q", "w", "s", "z"}.

After adding 1 typo, 6.90% of all the data samples were classified differently compared with the original predictions. Among all the samples that were correctly classified before adding any typos, only 95.09% of them can be correctly classified after adding 1 typo. The metrics worsen after adding 2 typos. After adding 2 typos, 10.85% changed the predictions, and among all the samples that were correctly classified before adding any typos, only 91.98% of them can be correctly classified. Here, I show 3 examples when adding 1 typo and 3 examples when adding 2 typos.

3 Examples when adding 1 typo:

1. Original Text: *RT @user Can I politely suggest that we all stop using the term "alt-right" and use other, more precise words.*
   Text w/ 1 typo: *RT @user Can I politely suggest that we all stop uwing the term "alt-right" and use other, more precise words.*
   True Label: NEU – Original Prediction: NEU – Prediction after adding 1 typo: NEG.

2. Original Text: *Ashley Graham needed affirmation after being dumped #bollywood*
   Text w/ 1 typo: *Ashley Graham needed affirmqtion after being dumped #bollywood*
   True Label: NEU – Original Prediction: NEU – Prediction after adding 1 typo: NEG.

3. Original Text: *Link in bio - Come on The Walking Dead, get it together. #TheWalkingDead*
   Text w/ 1 typo: *Link in bio - Come on The Walking Dead, get it together. #TheWalkinyDead*
   True Label: POS – Original Prediction: POS – Prediction after adding 1 typo: NEU.

3 examples when adding 2 typos:

1. Original Text: *Google Earth joins the VR revolution letting you wander the world in 3D:*
   Text w/ 2 typos: *Gkogle Earth joins the VR revolution letting you wanxer the world in 3D:*
   True Label: POS – Original Prediction: POS – Prediction after adding 2 typos: NEU.

2. Original Text: *Abortion really brings out the kid in you.*
   Text w/ 2 typos: *Abortioj really brings out yhe kid in you.*
   True Label: NEG – Original Prediction: NEG – Prediction after adding 2 typos: NEU.

3. Original Text: *Facebook has banned modeling pictures of Melania Knauss Trump*
   Text w/ 2 typos: *Facebook has bamned modeling pidtures of Melania Knauss Trump*
   True Label: NEG – Original Prediction: NEG – Prediction after adding 2 typos: NEU.

4.3 Ironic Text

Irony means that a statement's intended meaning is the opposite of its literal meaning. It is often used to express sarcasm, mockery, or criticism humorously. People tend to use this as a way to express negative sentiments, although the words are positive. For example, the sentence "*I just love it when my flight gets delayed for hours*" is very likely to be ironic, expressing frustration about the delayed flight.

The challenge with ironic text lies in its discrepancy between the surface-level meaning and the intended sentiment. To test the model's performance with ironic text, I asked GPT-4 to come up with 100 ironic sentences with negative sentiments, and I manually checked every one of them to make sure that they all ironically express negative sentiments.

Out of 100 ironic sentences with negative sentiments, 32 were classified as NEG, 0 were classified as NEU, and 68 were classified as POS. The performance is very bad and the accuracy is almost the same as the random guessing. Here, I list 5 examples that were wrongly classified:

1. *Oh great, another fantastic traffic jam. Just what I needed today!*

2. *Love when my coffee spills all over my desk.*

3. *Nothing makes me happier than getting a parking ticket.*

4. *Oh wonderful, my car won't start. I was hoping for some extra exercise.*

5. *Nothing beats the joy of being late to an important meeting.*

The above 5 sentences were all classified as positive sentiment.

4.4 Biases

Although sentiment analysis models are powerful, they are trained by humans and inevitably reflect human biases in their results. For example, this paper(Kiritchenko and Mohammad, 2018) evaluated 219 automatic sentiment analysis systems, and they found that more than 75% of the systems tend to mark sentences involving one gender/race with higher intensity scores than the sentences involving the other gender/race. The following sections will inspect model's biases regarding to gender/race, nationality/region, age, disability, and socioeconomic status.

4.4.1 Gender/Race

I used EEC dataset to test if this model presents biases regarding gender/race. This dataset has 4 labels: joy, fear, anger, and sadness. I classified joy as positive sentiments and others as negative sentiments.

	African	European	Male	Female
NEG	700	700	1050	1050
POS	2180	2180	3270	3270

Table 4: Number data samples of different categories

Table 4 shows the number of data samples of different categories, then we will compare the model's accuracy and softmax scores of different categories to see if it has biases regarding gender/race.

	African	European	Male	Female
Acc	85.46%	85.73%	85.93%	86.42%
Score	0.72	0.73	0.73	0.75

Table 5: Model Performance of negative sentiments, score means the average softmax score of negative class

	African	European	Male	Female
Acc	94.86%	95.86%	95.90%	95.23%
Score	0.89	0.90	0.90	0.90

Table 6: Model Performance of positive sentiments, score means the average softmax score of positive class

Table 5 and 6 show the model's accuracies and average softmax scores of negative and positive sentiments. We can see that the model performs only slightly better when sentences are related to European for both positive and negative sentiments than sentences related to African. For different genders, the model has better performance of female-related sentences for negative sentiments and better performance of male-related sentences for positive sentiments; however, the difference is still small. Therefore, I think we can conclude that the model has little biases regarding to gender/race.

4.4.2 Nationality/Region

Here, I designed some adversarial sentences to test if the model has biases regarding different nationalities/regions. The following are three pairs of sentences illustrating that the model has biases about nationality/region.
Example 1:

- *This car was made in Germany.*
  Model Prediction: `NEU`
- *This car was made in Afghanistan.*
  Model Prediction: `NEG`

Example 2:

- *A man from North America is holding a gun.*
  Model Prediction: `NEU`
- *A man from Middle East is holding a gun.*
  Model Prediction: `NEG`

Example 3:

- *USA wants to develop its economy and offers healthcare to everyone.*
  Model Prediction: `POS`
- *Iran wants to develop its economy and offers healthcare to everyone.*
  Model Prediction: `NEU`

For each pair of the above 3 pairs of sentences, everything is the same except the nationality/region. However, the model made different predictions. This can prove that this model has biases towards to different nationalities/regions.

4.4.3 Age & Disability & Socioeconomic Status

I also came up with some adversarial text to test if this model has biases regarding age, disability, or socioeconomic status. The following are 3 examples:
Age Example:

- *This company hires many young people.*
  Model Prediction: `NEU`

- *This company hires many old people.*
  Model Prediction: `NEG`

Disability Example:

- *The blind artist created a sculpture.*
  Model Prediction: `NEU`
- *The artist created a sculpture.*
  Model Prediction: `POS`

Socioeconomic Status Example:

- *A wealthy person entered a restaurant.*
  Model Prediction: `NEU`
- *A homeless person entered a restaurant.*
  Model Prediction: `NEG`

From the above 3 examples, we can say that the model exists biases regarding age, disability, and socioeconomic status

5 Discussion

5.1 Negation Words

Several works (Mukherjee et al., 2021), (Dadvar et al., 2011) have shown that negation words may cause misclassification in sentiment classification. I hypothesize the following reasons may cause misclassification:

1. The training data does not contain enough examples of sentences with negation, as a result, the model may not learn to handle negations correctly.
2. The model might not correctly identify the scope of the negation, leading to misinterpretation of the sentiment. For example, given text "*The movie was not bad*", the model may might think that "*not*" negates the the sentiment of the whole sentence instead of the the word "*bad*", which leads to the negative sentiment of the whole sentence.

5.2 Typos

RoBERTa-base model uses a word-level tokenizer, which I hypothesize is the main reason for performance drop when there are typos. I tested a simple word "*the*". If users spell it correctly, the tokenizer will treat it as one token. However, if users misspell it as "*yhe*", the tokenizer will consider the single character "*y*" as a token and "*he*" as another token. This paper (Shawky et al., 2024) also investigated how typos can affect the performance of text classification. It pointed out that the word-level tokenizer is less robust than the character-level tokenizer when increasing the percentage of typos.

5.3 Ironic Text

As shown in section 4.3, this model performs pretty badly when text is ironic. I hypothesize the following are the reasons for the bad performance:

1. Irony often involves using positive words to express negative sentiments or vice versa. The discrepancy between surface-level meaning and intended meaning can confuse the model.
2. This model is trained for sentiment classification on the sentence level; therefore, the model has less contextual content to leverage compared with paragraphs or passages. Lack of contextual content may also lead to misclassification of ironic text.

5.4 Biases

There are several papers (Mei et al., 2023), (Venkit and Wilson, 2021) explored the biases about sentiment classification task. These papers all show that many different kinds of biases exist in sentiment classification models, which match the findings presented in section 4.4. I hypothesize the reason for different kinds of biases is that this model was trained on around 124M tweets, and these tweets may have inherited some human biases or prejudices.

6 The way forward

6.1 Data Augmentation

From the aspect of data augmentation, I think we can do the following 3 things to improve the model performance:

1. For each training sentence, we can generate another training sentence with some random typos in it with the same label as the original text. By doing this, the model should be more robust to typos.
2. This paper (Mei et al., 2023) categorized 93 stigmatized conditions such that biases may happen. At the same time, they summarized 29 non-stigmatized conditions as the opposite of stigmatized conditions. For example, regarding ethnicity, Caucasian and European

American are considered as non-stigmatized conditions. Therefore, whenever we found training sentences that fall into one of the 29 non-stigmatized conditions, we can create corresponding sentences with stigmatized conditions with the same labels as the original text. I believe this can reduce the model biases. However, doing such data augmentation needs a lot of manpower; therefore, we may need to leverage the help of LLM or use templatic augmentations.

3. Ironic text may not be common if we collect data from Twitter, customer reviews, etc. Therefore, we may ask GPT to create some ironic text with sentiment labels for training model. Also, I have not found any dataset with ironic text and sentiment labels, which leads to the possibility of creating such a dataset.

I also found that LangTest Python library is suitable for the first two points mentioned above. It supports data augmentation about typos, gender bias, low-income country bias, ethnicity bias, and religion bias. In addition, it also allows users to customize templatic augmentations.

6.2 Add Spell Checker

We can apply spelling correction before feeding sentences into the model. Some potential Python libraries are pyspellchecker, TextBlob, and autocorrect. This may decrease the percentage of typos, thus improving the model performance.

6.3 Consider Negation Scopes

This paper (Mukherjee et al., 2021) mentioned that there are 4 types of negations, which are Morphological Negation, Syntactical Negation, Double Negation and Implicit Negation. This paper also summarizes the identification(how to identify each type of the negation) and the Scope of Negation for each type of negation.

For example, we can identify Syntactical Negation by looking for words such as "no", "not", "never", "no one", "neither", "none", etc. And the scope of negation is limited by the next punctuation mark or is extended or delimited by a compound Verb.

By using the above information, we can create negation equivalent tokens for each word that is in the scope of negation. This method is also mentioned in (Mukherjee et al., 2021). The negation equivalent tokens will have "_NEG" suffix appended. For example, if the input text is "*It is good.*", then the tokenizer may create 6 tokens as "⟨ *START* ⟩", "*It*", "*is*", "*good*", ".", "⟨ *END* ⟩" . If we change the input text to "*It is not good.*", then the tokenizer will create 7 tokens as "⟨ *START* ⟩", "*It*", "*is*", "*not*", "*good_NEG*", ".", "⟨ *END* ⟩" . We hope that the model can be more sensitive to negations by explicitly creating negation equivalent tokens. However, the drawback is the increase of the number of tokens.

6.4 Leverage the help of Irony Detection Models

There are many models trained to identify irony in the text. For example, there is another model trained from tweets to detect irony(Twitter-roBERTa-base model for Irony Detection). Irony detection models will output a probability of being ironic. Using this irony detection model, it can successfully classify all the sentences used in section 4.3 as ironic. If the irony detection model outputs a high probability of containing irony, we can output the opposite of the original prediction if the original prediction is `POS` or `NEG`. The theory behind this is that an ironic statement's intended meaning is the opposite of its literal meaning. The algorithm can be described as the following. Let t represent the text, $model_{sc}$ represent the sentiment classification model and $model_{id}$ represent the irony detection model.

Algorithm 1 Sentiment Classification with the help of Irony Detection

function SC(t, $model_{sc}$, $model_{id}$, thr_{irony})
  $label \leftarrow model_{sc}(t)$
  $p_{irony} \leftarrow model_{id}(t)$
  if $p_{irony} > thr_{irony}$ **then**
    if $label ==$ `POS` **then**
      $label =$ `NEG`
    else if $label ==$ `NEG` **then**
      $label =$ `POS`
    end if
  end if
  return $label$
end function

In the above algorithm, thr_{irony} is the threshold of being ironic, this can be treated as a hyperparameter. The above algorithm simply returns the opposite sentiment if the sentence is ironic. An alternative solution is to reduce the softmax score

of the original predicted class by a certain amount and then return the class with the highest softmax score. This alternative solution can ensure that the model will return the original prediction if it's very confident about it.

6.5 Model

Currently, RoBERTa-base model has 12 attention heads and the token embedding size is 768 with a total of 123M parameters. This paper (Loureiro et al., 2022) also pre-trained a RoBERTa-large model with 16 attention heads, 1024 token embedding size, and in total 355M parameters using tweets. We can fine-tune this RoBERTa-large model on the TweetEval dataset for sentiment classification, and hopefully larger number of heads and token embedding size can give the sentiment classification model a better contextual understanding.

6.6 Do not waste the test data

As a sentiment classification model, domain shift is common. Here, I list two possible causes for the domain shift:

- Different usage scenarios may have different language patterns, vocabularies, etc. For example, tweets and Amazon product reviews may have different language patterns.
- Language changes over time. The same words take on new meanings as time changes, and people develop different language patterns.

Therefore, we can use test time data to continue to train the model. To do this, we need to pick data samples with low confidence during the test time and use these data samples to continue to train the model. For example, given a sentence, the softmax scores for `POS`, `NEU`, `NEG` are 40%, 30%, and 30%, although the model outputs `POS`, the uncertainty is not small. Then, we can use this data sample to train the model. To train the model using test time data samples, we need labels. These labels could be pseudo-labels that are predicted by the model. An example paper is CoTTA (Wang et al., 2022). Also, we can let humans label these data samples with low confidence. An example paper is ATTA (Gui et al., 2024).

7 Implementation

I did not do this part.

8 Conclusion

This analysis study presents 4 limitations of a RoBERTa-base sentiment classification model. 3 of them are undesirable performances when sentences have negation words, typos, and ironic contents. The 4th limitation is that this model contains biases regarding nationality/region, age, disability, and socioeconomic status. Section 6 proposed a comprehensive plan in terms of data, algorithm, and model to improve the current sentiment classification model.

9 Acknowledgements

I only used GPT4 to generate 100 ironic sentences as adversarial data samples to test the model. The details can be found in section 4.3. Everything is completed by myself.

References

Barbieri, F., Camacho-Collados, J., Neves, L., and Espinosa-Anke, L. (2020). Tweeteval: Unified benchmark and comparative evaluation for tweet classification. *arXiv preprint arXiv:2010.12421*.

Dadvar, M., Hauff, C., and De Jong, F. M. (2011). Scope of negation detection in sentiment analysis. In *11th Dutch-Belgian Information Retrieval Workshop, DIR 2011*, pages 16–20. University of Amsterdam.

Gui, S., Li, X., and Ji, S. (2024). Active test-time adaptation: Theoretical analyses and an algorithm. *arXiv preprint arXiv:2404.05094*.

Kiritchenko, S. and Mohammad, S. M. (2018). Examining gender and race bias in two hundred sentiment analysis systems. *arXiv preprint arXiv:1805.04508*.

Loureiro, D., Barbieri, F., Neves, L., Anke, L. E., and Camacho-Collados, J. (2022). Timelms: Diachronic language models from twitter. *arXiv preprint arXiv:2202.03829*.

Mei, K., Fereidooni, S., and Caliskan, A. (2023). Bias against 93 stigmatized groups in masked language models and downstream sentiment classification tasks. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency*, pages 1699–1710.

Mukherjee, P., Badr, Y., Doppalapudi, S., Srinivasan, S. M., Sangwan, R. S., and Sharma, R. (2021). Effect of negation in sentences on sentiment analysis and polarity detection. *Procedia Computer Science*, 185:370–379.

Shawky, P. E., ElKaffas, S. M., and Guirguis, S. K. (2024). Effect of typos on text classification accuracy in word and character tokenization. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 40(2):152–162.

Venkit, P. N. and Wilson, S. (2021). Identification of bias against people with disabilities in sentiment analysis and toxicity detection models. *arXiv preprint arXiv:2111.13259*.

Wang, Q., Fink, O., Van Gool, L., and Dai, D. (2022). Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7201–7211.

Wankhade, M., Rao, A. C. S., and Kulkarni, C. (2022). A survey on sentiment analysis methods, applications, and challenges. *Artificial Intelligence Review*, 55(7):5731–5780.